Department of Computer and Software Engineering
SE: Machine Learning

Course Instructor: Dr. Ahmed Raza	**Date:**
Day:	**Semester:** 6th

Lab 6: K-Means Clustering from Scratch

Lab Title	CLO	BT	PLO	Weightage
K-Means Clustering From Scratch				

Name	Roll Number	Report /10	Viva /5	Total /15
Muhammad Hamza Arshad	BSSE23072			

Checked on: _____

Signature: _____

1 Learning Outcomes

After completing this lab students will be able to:

- Implement the K-Means clustering algorithm from scratch.
- Understand how different distance metrics affect clustering.
- Compare clustering results on separable vs overlapping datasets.
- Visualize cluster decision boundaries.

2 Datasets

In this lab we will experiment with two datasets.

2.1 Dataset 1: Iris Dataset

The Iris dataset contains 150 samples. For visualization purposes we only use two features:

- Sepal Length
- Petal Length

2.2 Dataset 2: Synthetic Dataset

We will also generate a synthetic dataset using:

`sklearn.datasets.make_blobs`

Two versions will be created:

- Well-separated clusters
- Overlapping clusters

This allows us to observe how K-Means behaves when clusters are not clearly separable.

3 Distance Metrics

In this lab you will experiment with three distance metrics.

3.1 Euclidean Distance

$$d(x, y) = \sqrt{\sum (x_i - y_i)^2}$$

Produces circular cluster boundaries.

3.2 Manhattan Distance

$$d(x, y) = \sum |x_i - y_i|$$

Produces diamond-shaped cluster boundaries.

3.3 Cosine Distance

Cosine similarity measures the angle between two vectors.

$$\text{similarity} = \frac{x \cdot y}{||x||\,||y||}$$

Cosine distance:

$$d(x, y) = 1 - similarity$$

This metric is commonly used in:

- Text clustering
- Document similarity
- Embedding spaces

4 Part A: Distance Function (3 Marks)

Implement the function:

`_distance(x1, x2)`

The function must support Euclidean, Manhattan, and Cosine metrics. Use NumPy operations only.

5 Part B: Cluster Assignment (4 Marks)

Implement:

`assign_clusters(X, centroids)`

Steps:

- Compute the distance from each point to every centroid
- Assign the point to the closest centroid
- Return cluster labels

6 Part C: Update Centroids (4 Marks)

Implement:

`update_centroids(X, labels)`

Steps:

- For each cluster compute the mean of all assigned points
- Return updated centroid coordinates

7 Part D: Full K-Means Algorithm (4 Marks)

Implement:

```
fit(X)
predict(X)
```

Algorithm:

1. Initialize K centroids randomly
2. Assign points to nearest centroid
3. Update centroid locations
4. Repeat until centroids converge or max iterations reached

8 Part E: Boundary Visualization (Bonus)

Create a function:

`plot_decision_boundaries(model, X)`

Run the visualization for Euclidean, Manhattan, and Cosine distances.

9 Part F: Experimental Comparison

Run K-Means on both datasets.

Dataset	Distance Metric	Observations
Iris	Euclidean	Boundaries are smooth, straight lines (perpendicular bisectors). The algorithm assumes clusters are perfect spheres radiating from the centroid.
Iris	Manhattan	Boundaries are jagged and stair-stepped. Movement is restricted to grid lines, forcing orthogonal, diamond-shaped decision regions.
Iris	Cosine	Boundaries radiate outward from the origin $(0, 0)$ like pie slices. Groups points by trajectory/angle rather than physical magnitude. Performs very poorly on physical measurements like petal length.
Separable Blobs	Euclidean	Perfect separation. When clusters have low variance and are far apart, K-Means easily identifies the distinct spherical distributions.
Overlapping Blobs	Euclidean	Fails to capture true variance. K-Means forces rigid, linear boundaries directly through dense shared regions, misclassifying border points because it lacks density awareness.

Discussion Questions:

- **How do distance metrics change cluster shapes?**
  Answer: The geometric reality of "closest" shifts. Euclidean yields Voronoi cells (straight lines bounding circular clusters). Manhattan restricts distance to orthogonal grids, creating jagged, diamond-like boundaries. Cosine ignores distance entirely and splits the space based on angular trajectory.

- **Which metric performs best for overlapping clusters?**
  Answer: None of the standard metrics perform perfectly because K-Means assumes spherical clusters and equal variance. It forces hard, rigid boundaries regardless of the metric. However, Euclidean provides the most stable baseline partition. True overlapping data requires probabilistic density models (like Gaussian Mixture Models), not K-Means.
- **Does K-Means always converge to the same solution?**
  Answer: No. The final centroids are highly dependent on the initial random placement of the starting centroids. Without setting a fixed random seed, K-Means can easily converge to different local minima.

10 Cluster Quality: Within Cluster Sum of Squares

K-Means attempts to minimize the objective function (Inertia/WCSS):

$$J = \sum_{i=1}^{n} ||x_i - \mu_{c_i}||^2$$

Lower values indicate tighter clusters.

11 Choosing the Number of Clusters: Elbow Method

The Elbow Method helps estimate a reasonable value for K.

12 Part F: Compute Inertia (3 Marks)

Implement the function:

```
compute_inertia(X, labels, centroids)
```

13 Part G: Elbow Method (Bonus)

Create a function:

```
plot_elbow_curve(X, k_values)
```

Which value of K appears optimal for the dataset?
Answer: Based on the generated elbow curve, the optimal value is $K = 2$ **or** $K = 3$. The inertia drops massively from $K = 1$ to $K = 2$, and significantly again to $K = 3$. After $K = 3$, the curve flattens out, meaning adding more clusters yields diminishing returns in variance reduction.

Appendix: Laboratory Visualizations

Note: The following decision boundaries and curves were generated using the custom MyKMeans implementation on Apple Silicon Mac and my environment is uv.

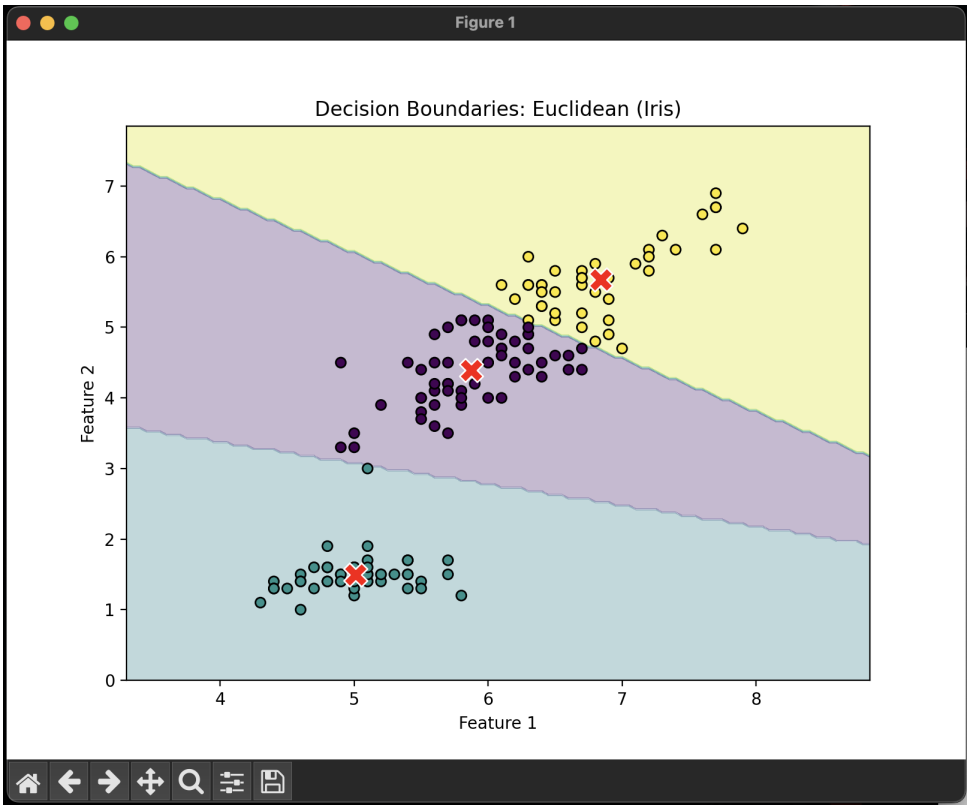

Figure 1: Decision Boundaries: Euclidean (Iris Dataset)

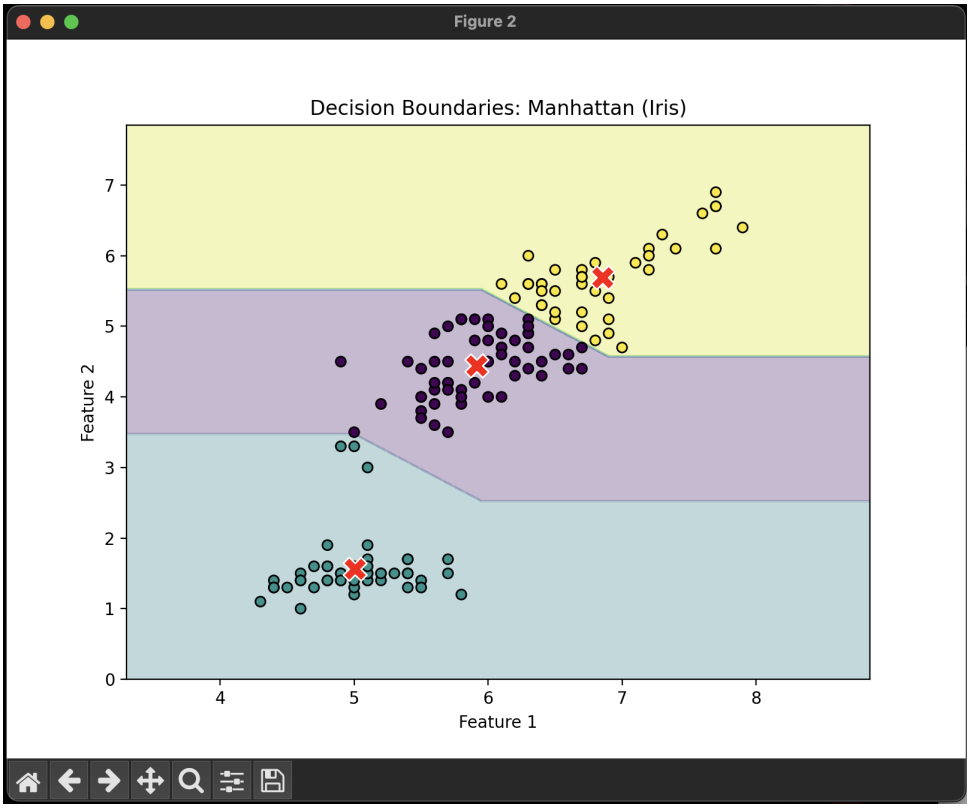

Figure 2: Decision Boundaries: Manhattan (Iris Dataset)

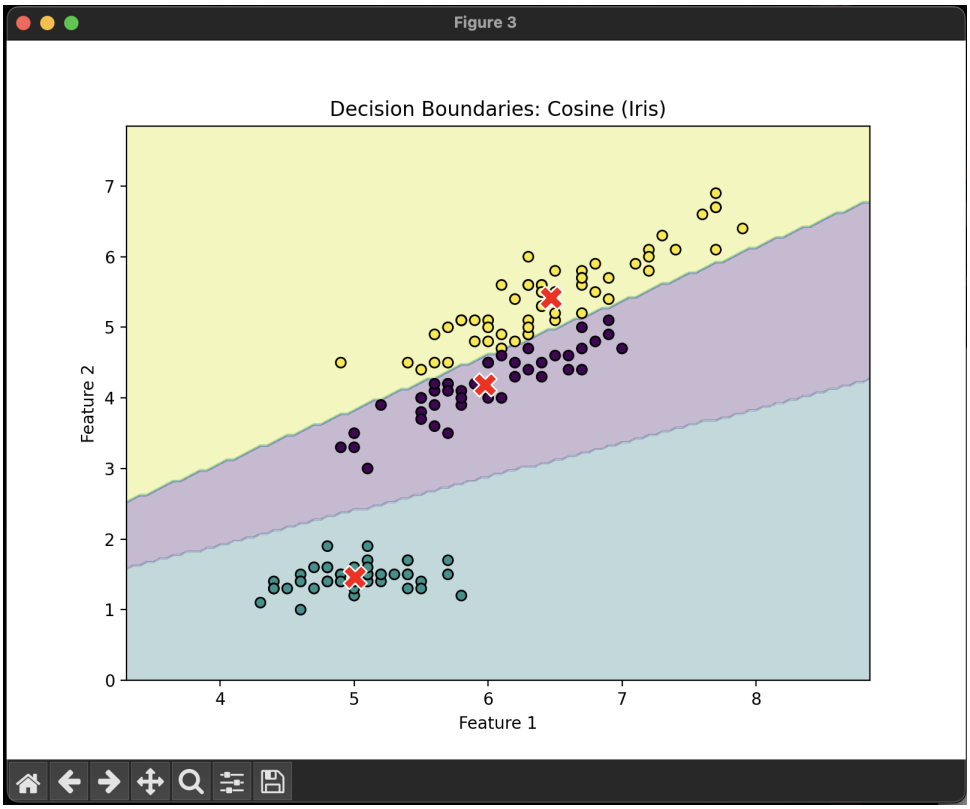

Figure 3: Decision Boundaries: Cosine (Iris Dataset)

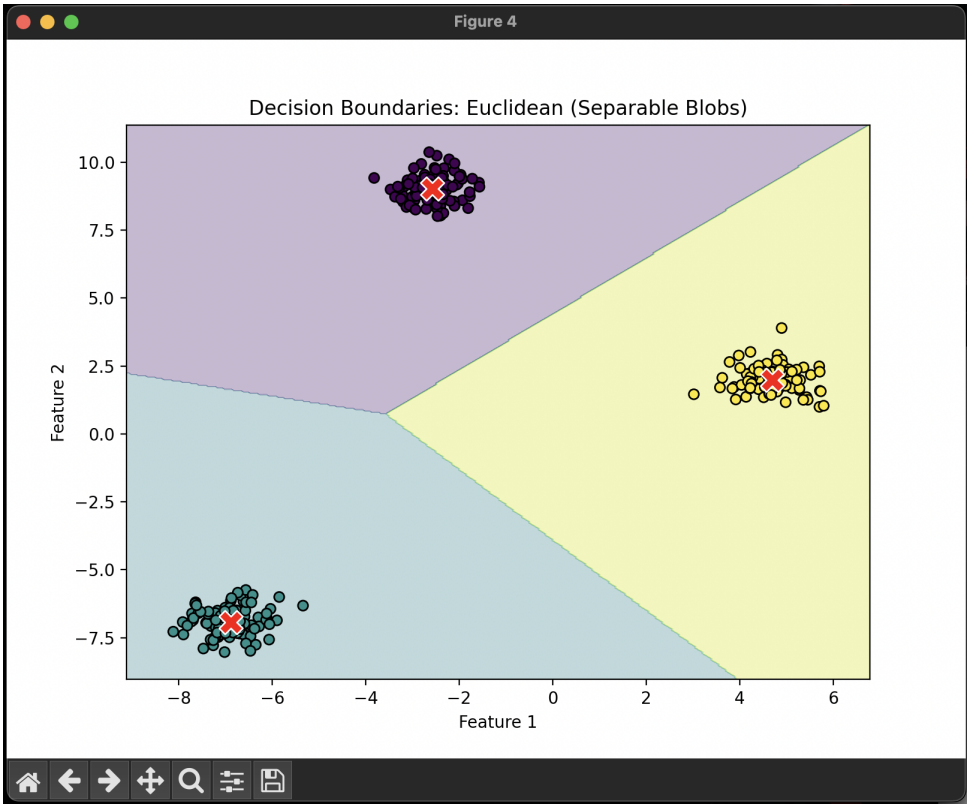

Figure 4: Decision Boundaries: Euclidean (Separable Blobs)

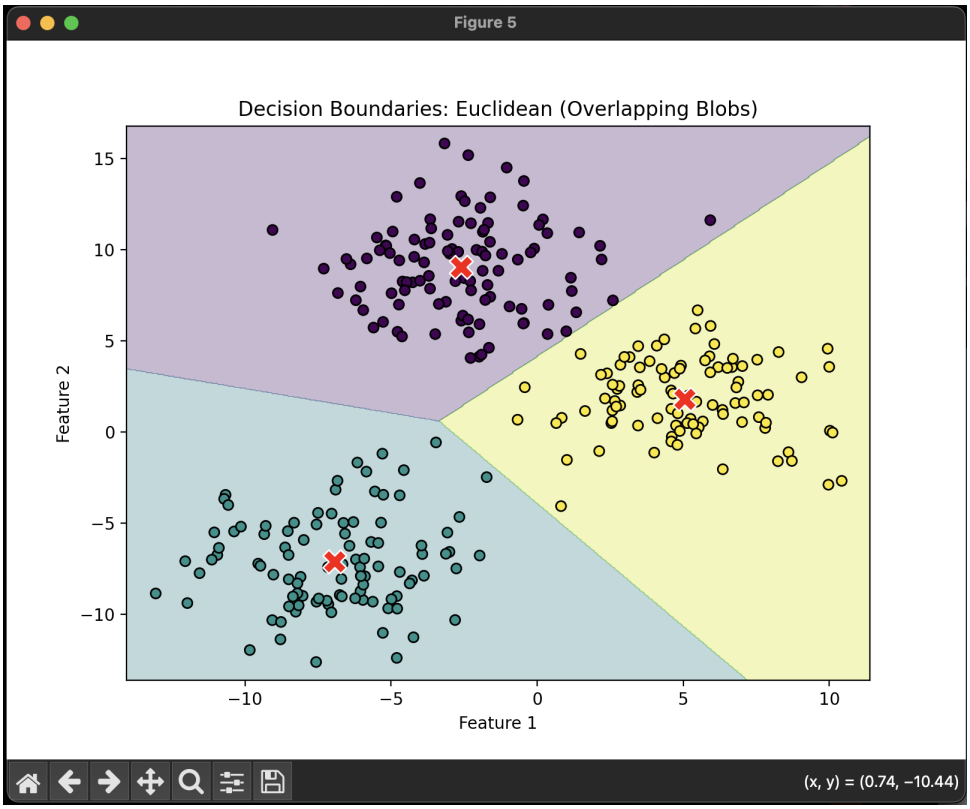

Figure 5: Decision Boundaries: Euclidean (Overlapping Blobs)

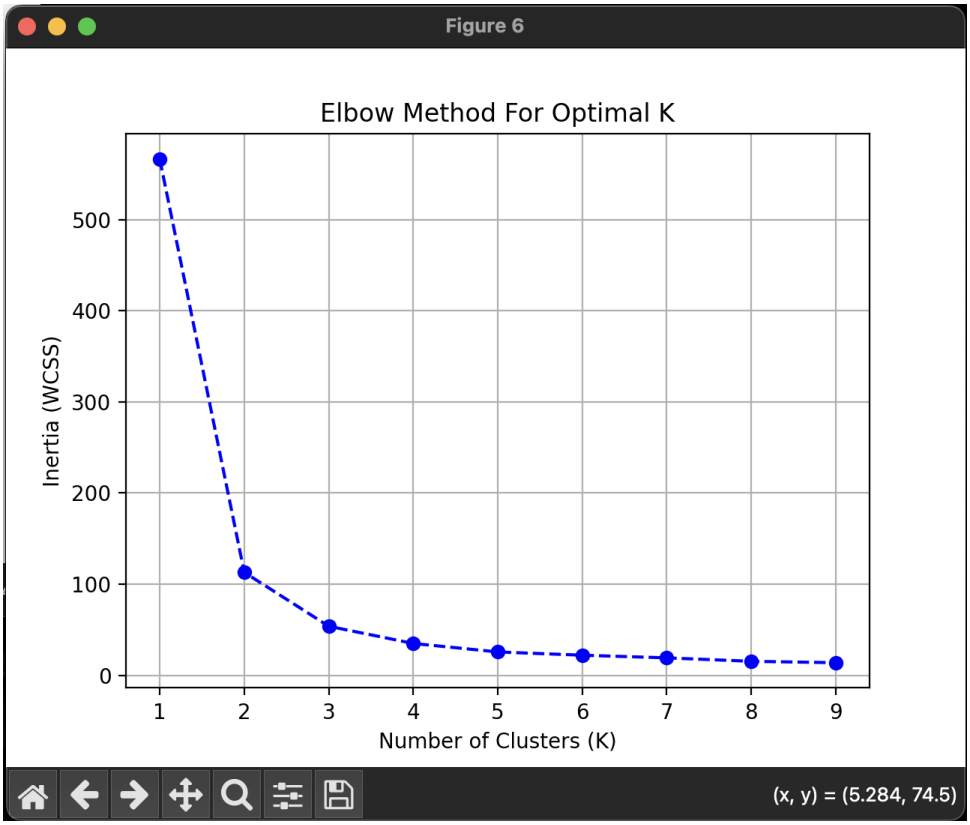

Figure 6: Elbow Method for Optimal K